

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1
По дисциплине: «Криптография и сетевая безопасность»

Выполнил:
Студент 4-го курса
Группы АС-66
Николова М.С.
Проверил:
Хацкевич А.С.

Цель работы: освоить программную реализацию и практическое применение алгоритмов генерации псевдослучайных последовательностей (РСП), методов их статистической оценки, а также вероятностных и детерминированных тестов простоты чисел с построением доказуемо простых чисел.

Вариант 6
Ход работы

Задание 1

1. Согласно варианту реализовать приложение, генерирующие псевдослучайную равномерно распределенную последовательность произвольной длины из заданного алфавита.
2. Подобрать параметры данного генератора таким образом, чтобы период последовательности имел максимальное значение.
3. Используя статистическое тестирование, проверить гипотезу о том, что сгенерированная последовательность действительно имеет равномерное распределение (см. приложение 2).
4. Отчет должен обязательно содержать несколько тестов, отображающих изменение статистических свойств последовательностей в зависимости от параметров генератора.

Вариант	Генератор	Размер алфавита	Дополнение
6	Линейный конгруэнтный генератор	N=15	<i>Рассмотреть следующие случаи:</i> 1. Мультипликативный генератор Лемера: $c = 0$ 2. Смешанный генератор

```
package Part1;  
import java.util.Arrays;
```

```
public class Lenear {  
  
    public static void main(String[] args) {  
        int N = 15;  
  
        System.out.println("1. Мультипликативный генератор Лемера (c = 0)");  
  
        testMultiplicative(1, 2, N, 10000);  
  
        System.out.println("\n2. Смешанный генератор (c != 0), теорема Халла-Доббелла");  
  
        testMixed(0, 16, 4, N, 10000);  
  
        System.out.println("\n3. Смешанный генератор с параметрами (для сравнения)");  
        testMixed(0, 3, 3, N, 10000);  
    }  
}
```

```

public static void testMultiplicative(int x0, int a, int N, int count) {
    int[] sequence = generate(x0, a, 0, N, count);
    System.out.println("Первые 20 элементов: " + Arrays.toString(Arrays.copyOf(sequence, 20)));

    int period = findPeriod(x0, a, 0, N);
    int lambda = carmichaelLambda(N);
    boolean primitiveRootExists = (lambda == eulerPhi(N));

    System.out.println("Период генератора: " + period);
    System.out.println("Функция Кармайкла lambda(" + N + ") = " + lambda
        + (primitiveRootExists ? " (совпадает с phi(N), первообразные корни существуют)"
        : " (первообразных корней по модулю " + N + " не существует, "
        + "т.к. " + N + " не имеет вида 2, 4, p^k или 2*p^k)"));
    System.out.println("=> Теоретически достижимый максимум периода для МКГ: " + lambda);

    checkChiSquare(sequence, N, period);
}

public static void testMixed(int x0, int a, int c, int N, int count) {
    int[] sequence = generate(x0, a, c, N, count);
    System.out.println("Первые 20 элементов: " + Arrays.toString(Arrays.copyOf(sequence, 20)));

    int period = findPeriod(x0, a, c, N);
    System.out.println("Период генератора: " + period + " (макс. возможный : " + N + ")");

    HullDobellResult check = checkHullDobell(a, c, N);
    System.out.println("Проверка условий :");
    System.out.println(" а) НОД(c, N) = 1 : " + check.gcdOk);
    System.out.println(" б) (a-1) кратно всем простым делителям N : " + check.primeFactorsOk);
    System.out.println(" в) (a-1) кратно 4, если N кратно 4 : " + check.mod4Ok
        + (N % 4 != 0 ? " (условие неприменимо, N не кратно 4)" : ""));
    System.out.println(" ИТОГ: условия теоремы " + (check.allOk ? "ВЫПОЛНЕНЫ -> ожидается T = N"
        : "НЕ выполнены -> период короче N"));

    checkChiSquare(sequence, N, period);
}

public static int[] generate(int x0, int a, int c, int N, int count) {
    int[] sequence = new int[count];
    int current = x0;
    for (int i = 0; i < count; i++) {
        current = (a * current + c) % N;
        sequence[i] = current;
    }
    return sequence;
}

public static int findPeriod(int x0, int a, int c, int N) {
    int slow = (a * x0 + c) % N;
    int fast = (a * ((a * x0 + c) % N) + c) % N;

```

```

while (slow != fast) {
    slow = (a * slow + c) % N;
    fast = (a * ((a * fast + c) % N) + c) % N;
}

```

```

int steps = 1;
fast = (a * slow + c) % N;
while (slow != fast) {
    fast = (a * fast + c) % N;
    steps++;
}
return steps;
}

```

```

public static void checkChiSquare(int[] sequence, int N, int period) {
    int[] frequencies = new int[N];
    for (int val : sequence) {
        frequencies[val]++;
    }

```

```

double expected = (double) sequence.length / N;
double chiSquare = 0.0;
for (int i = 0; i < N; i++) {
    double diff = frequencies[i] - expected;
    chiSquare += (diff * diff) / expected;
}

```

```

double criticalValue = 23.685;

```

```

System.out.println("Частоты выпадения символов [0.." + (N - 1) + "]: " + Arrays.toString(frequencies));
System.out.printf("Значение Хи-квадрат (наблюдаемое): %.3f\n", chiSquare);
System.out.println("Критическое значение (alpha=0.05, df=14): " + criticalValue);

```

```

if (chiSquare <= criticalValue) {
    System.out.println("ИТОГ: Гипотеза о равномерном распределении ПОДТВЕРЖДАЕТСЯ.");
} else {
    System.out.println("ИТОГ: Гипотеза о равномерном распределении ОТКЛОНЯЕТСЯ.");
    if (period < N) {
        System.out.println("Пояснение: период генератора (" + period + ") меньше размера алфавита ("
            + N + "), поэтому последовательность физически не может принимать все "
            + N + " значений - отклонение гипотезы здесь ОЖИДАЕМО, а не свидетельствует об ошибке.");
    }
}
}

```

```

private static int gcd(int x, int y) {
    return y == 0 ? x : gcd(y, x % y);
}

```

```

private static int eulerPhi(int N) {
    int result = N;
    int n = N;
    for (int p = 2; (long) p * p <= n; p++) {
        if (n % p == 0) {
            while (n % p == 0) n /= p;
            result -= result / p;
        }
    }
    if (n > 1) result -= result / n;
    return result;
}

```

```

private static int carmichaelLambda(int N) {
    int n = N;
    int lambda = 1;
    for (int p = 2; (long) p * p <= n; p++) {
        if (n % p == 0) {
            int exp = 0;
            while (n % p == 0) { n /= p; exp++; }
            int pk = (int) Math.pow(p, exp);
            int lambdaPk;
            if (p == 2 && exp >= 3) {
                lambdaPk = pk / 4;
            } else {
                lambdaPk = pk - pk / p;
            }
            lambda = lcm(lambda, lambdaPk);
        }
    }
    if (n > 1) {
        lambda = lcm(lambda, n - 1);
    }
    return lambda;
}

```

```

private static int lcm(int x, int y) {
    return x / gcd(x, y) * y;
}

```

```

private static java.util.List<Integer> primeFactors(int N) {
    java.util.List<Integer> factors = new java.util.ArrayList<>();
    int n = N;
    for (int p = 2; (long) p * p <= n; p++) {
        if (n % p == 0) {
            factors.add(p);
            while (n % p == 0) n /= p;
        }
    }
    if (n > 1) factors.add(n);
    return factors;
}

```

```

private static class HullDobellResult {
    boolean gcdOk, primeFactorsOk, mod4Ok, allOk;
}

private static HullDobellResult checkHullDobell(int a, int c, int N) {
    HullDobellResult r = new HullDobellResult();
    r.gcdOk = gcd(c, N) == 1;

    r.primeFactorsOk = true;
    for (int p : primeFactors(N)) {
        if ((a - 1) % p != 0) {
            r.primeFactorsOk = false;
            break;
        }
    }

    r.mod4Ok = (N % 4 != 0) || ((a - 1) % 4 == 0);

    r.allOk = r.gcdOk && r.primeFactorsOk && r.mod4Ok;
    return r;
}
}

```

=== ЗАДАНИЕ 1. Линейный конгруэнтный генератор, N = 15 ===

--- 1. Мультипликативный генератор Лемера (с = 0) ---

Первые 20 элементов: [2, 4, 8, 1, 2, 4, 8, 1, 2, 4, 8, 1, 2, 4, 8, 1, 2, 4, 8, 1]

Период генератора: 4

Функция Кармайкла $\lambda(15) = 4$ (первообразных корней по модулю 15 не существует, т.к. 15 не имеет вида $2, 4, p^k$)

=> Теоретически достижимый максимум периода для МКГ: 4

Частоты выпадения символов [0..14]: [0, 2500, 2500, 0, 2500, 0, 0, 0, 2500, 0, 0, 0, 0, 0, 0]

Значение Хи-квадрат (наблюдаемое): 27500,000

Критическое значение ($\alpha=0.05$, $df=14$): 23.685

ИТОГ: Гипотеза о равномерном распределении ОТКЛОНЯЕТСЯ.

Пояснение: период генератора (4) меньше размера алфавита (15), поэтому последовательность физически не может принимать все возможные значения.

--- 2. Смешанный генератор (с != 0), полный период по т. Халла-Доббелла ---

Первые 20 элементов: [4, 8, 12, 1, 5, 9, 13, 2, 6, 10, 14, 3, 7, 11, 0, 4, 8, 12, 1, 5]

Период генератора: 15 (макс. возможный : 15)

Проверка условий :

а) $\text{НОД}(c, N) = 1$: true

б) (a-1) кратно всем простым делителям N : true

с) (a-1) кратно 4, если N кратно 4 : true (условие неприменимо, N не кратно 4)

ИТОГ: условия теоремы ВЫПОЛНЕННЫ -> ожидается T = N

Частоты выпадения символов [0..14]: [666, 667, 667, 666, 667, 667, 667, 666, 667, 667, 667, 666, 667, 667, 666]

Значение Хи-квадрат (наблюдаемое): 0,005

Критическое значение ($\alpha=0.05$, $df=14$): 23.685

ИТОГ: Гипотеза о равномерном распределении ПОДТВЕРЖДАЕТСЯ.

Задание 2

1. Реализовать приложение, позволяющее генерировать простое число по следующей схеме (с использованием, например, теста Рабина - Миллера):

1. Сгенерируйте случайное p-битовое число n.

2. Установите старший и младший биты равными 1. (Старший бит гарантирует требуемую длину простого числа, а младший бит обеспечивает его нечетность.)

3. Убедитесь, что n не делится на небольшие простые числа: 3, 5, 7, 11, и т.д. Во

многих реализациях проверяется делимость p на все простые числа, меньшие 256. Наиболее эффективной является проверка на делимость для всех простых чисел, меньших 2000.

Все простые числа, не превосходящие 256 перечислены ниже:

2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,
83,89,97,101,103,107,109, 113,127,131,137,139,149,151,157,
163,167,173,179,181,191,193,197,199,211,223,227,229,233,239,
241,251.

4. Выполните один тест Рабина - Миллера проверки простоты числа для некоторого случайного a . Если p проходит тест, сгенерируйте другое случайное a и повторите проверку.

Выбирайте небольшие значения a для ускорения вычислений.

Выполните пять тестов. (Одного может показаться достаточным, но выполните пять.) Если p не проходит одной из проверок, сгенерируйте другое p и попробуйте снова.

Можно не генерировать и случайным образом каждый раз, но последовательно перебирать числа, начиная со случайно выбранного до тех пор, пока не будет найдено простое число.

Этап (3) не является обязательным, но это хорошая идея. Проверка, что случайное нечетное p не делится на 3, 5 и 7 отсекает 54 процента нечетных чисел еще до этапа (4). Проверка делимости на все простые числа, меньшие 100, убирает 76 процентов нечетных чисел, проверка делимости на все простые числа, меньшие 256, убирает 80 процентов нечетных чисел. В общем случае, доля нечетных кандидатов, которые не делятся ни на одно простое число, меньшее z , равна $1.12 / \ln z$.

Вместо теста Рабина - Миллера следует использовать тест, заданный в варианте.

2. В отчете обязательно должны присутствовать тесты, подтверждающие правильность разработанных программ.

Вариант	Алгоритм	Дополнение
6	Тест Леманна.	

```
package Part2;
```

```
import java.math.BigInteger;  
import java.util.Random;
```

```
public class Part2 {
```

```
    private static final int[] SMALL_PRIMES = {  
        2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79,  
        83, 89, 97, 101, 103, 107, 109, 113, 127, 131, 137, 139, 149, 151, 157, 163, 167,  
        173, 179, 181, 191, 193, 197, 199, 211, 223, 227, 229, 233, 239, 241, 251  
    };
```

```
    private static long lcgState = System.nanoTime() & 0xFFFL;  
    private static final long A = 1664525;
```

```

private static final long C = 1013904223;
private static final long N = 256;

private static int nextLcgByte() {
    lcgState = (A * lcgState + C) % N;
    return (int) lcgState;
}

public static BigInteger findPrime(int bitLength, int iterationsK) {
    System.out.println("Поиск простого числа (Тест Лемана)");
    System.out.println("Разрядность: " + bitLength + " бит, Количество проверок (k): " + iterationsK);

    long startTime = System.currentTimeMillis();
    int totalCandidates = 0;
    int filteredBySmallPrimes = 0;
    int failedLehmann = 0;

    while (true) {
        totalCandidates++;

        BigInteger candidate = generateCandidate(bitLength);

        if (!isPassedTrialDivision(candidate)) {
            filteredBySmallPrimes++;
            continue;
        }

        if (isPassedLehmannTest(candidate, iterationsK)) {
            long elapsedTime = System.currentTimeMillis() - startTime;

            System.out.println("\n--- РЕЗУЛЬТАТ НАЙДЕН ---");
            System.out.println("Сгенерированное простое число n: " + candidate);
            System.out.println("Всего проверено: " + totalCandidates);
            System.out.println("Отсеяно на этапе просеивания: " + filteredBySmallPrimes +
                String.format(" (%.1f%%)", (filteredBySmallPrimes * 100.0 / totalCandidates)));
            System.out.println("Отбраковано тестом Лемана: " + failedLehmann);
            System.out.println("Время работы: " + elapsedTime + " мс");
            return candidate;
        } else {
            failedLehmann++;
        }
    }
}

public static void main(String[] args) {
    findPrime(64, 5);
}

public static BigInteger generateCandidate(int bitLength) {
    byte[] bytes = new byte[(bitLength + 7) / 8];
    for (int i = 0; i < bytes.length; i++) {
        bytes[i] = (byte) (nextLcgByte() & 0xFF);
    }
    BigInteger n = new BigInteger(1, bytes);

```

```

    n = n.setBit(bitLength - 1);
    n = n.setBit(0);

    return n;
}

public static boolean isPassedTrialDivision(BigInteger n) {
    for (int p : SMALL_PRIMES) {
        BigInteger bigP = BigInteger.valueOf(p);
        if (n.equals(bigP)) return true;
        if (n.mod(bigP).equals(BigInteger.ZERO)) return false;
    }
    return true;
}

public static boolean isPassedLehmannTest(BigInteger n, int k) {
    if (n.compareTo(BigInteger.valueOf(3)) <= 0) return n.compareTo(BigInteger.ONE) > 0;

    BigInteger nMinusOne = n.subtract(BigInteger.ONE);
    BigInteger exponent = nMinusOne.divide(BigInteger.valueOf(2));
    BigInteger negativeOne = nMinusOne;

    Random rng = new Random();
    boolean hasMinusOne = false;

    for (int i = 0; i < k; i++) {
        BigInteger a;
        do {
            a = new BigInteger(n.bitLength(), rng);
        } while (a.compareTo(BigInteger.ONE) <= 0 || a.compareTo(n) >= 0);

        BigInteger t = a.modPow(exponent, n);

        if (!t.equals(BigInteger.ONE) && !t.equals(negativeOne)) {
            return false;
        }
        if (t.equals(negativeOne)) {
            hasMinusOne = true;
        }
    }
    return hasMinusOne;
}
}

```

=== ЗАДАНИЕ 2. Генерация простого числа (тест Лемана) ===

Поиск простого числа (Тест Лемана)

Разрядность: 64 бит, Количество проверок (k): 5

--- РЕЗУЛЬТАТ НАЙДЕН ---

Сгенерированное простое число n: 12325812801194039717

Всего проверено: 20

Отсеяно на этапе просеивания: 16 (80,0%)

Отбраковано тестом Лемана: 3

Время работы: 2 мс